Predicting hospital readmissions: a Machine Learning approach

Adam C. Maćkowiak-Pellón*
Instituto de Física Interdisciplinar y Sistemas Complejos (CSIC-UIB), Campus UIB, Spain
(Dated: April 21, 2026)

Hospital readmissions, particularly those within 30 days of discharge, present significant concerns for healthcare systems in terms of both patient outcomes and financial costs. In this study, we explore the application of classical machine learning techniques to predict hospital readmission risks using a dataset of 5000 patient encounters. Building upon the foundational work of Strack et al., we conduct extensive data preprocessing, including dimensionality reduction and feature engineering, followed by feature selection using Random Forest-based methods. We then compare the performance of several classification algorithms—including K-Nearest Neighbors, Decision Trees, Random Forest, Naive Bayes, Support Vector Machines, Logistic Regression, and XGBoost—evaluating them based on accuracy, cost, and AUC scores. Our findings highlight XGBoost and Random Forest as the most effective models for predicting readmissions, particularly in minimizing misclassification costs.

I. INTRODUCTION

Hyperglycemia, defined as elevated blood glucose levels, poses a considerable threat to both diabetic and non-diabetic individuals by heightening the risk of adverse health outcomes, including increased morbidity and mortality. With the emergence of extensive electronic health records and clinical databases, there is now greater potential to apply sophisticated machine learning methods to support medical decision-making and improve patient care. This research expands on the foundational work of Strack et al. [1], who explored the influence of HbA1c testing on hospital readmission rates, but also in the different machine learning algorithms explained during the course. In this study, we study 5000 cases insipired from the originial dataset to find patterns and predictive factors associated with hospital readmission within 30 days after discharge.

II. DATA EXPLORATION AND DATA PROCESSING

We analyzed a dataset of 5,000 medical encounters with 37 features (see Table I). This dataset is a subset of a larger one [2], which is also used in [1]. A more extensive explanation of each of these features can be found in both of the aforementioned references. Nevertheless, as we will see, some of these features require closer attention due to various factors.

The first step we took was decoding the features marked with an asterisk in Table I and grouping the data into a smaller set of categories. Machine learning algorithms such as KNN and SVM require categorical variables to be encoded numerically, as they cannot process raw categorical data directly. Additionally, reducing the number of categories simplifies the analysis. The grouping process for the features *admission_type_id*, *discharge_disposition_id*, and *admission_source_id* is presented in Table IV in the Appendix. In these cases, some categories were very similar, so we merged them to further reduce dimensionality.

Feature	Data Type
Unnamed: 0	Numerical
patient_nbr	Numerical
race	Nominal
gender	Nominal
age	Nominal
admission_type_id	Nominal*
discharge_disposition_id	Nominal*
admission_source_id	Nominal*
time_in_hospital	Numerical
num_lab_procedures	Numerical
num_procedures	Numerical
num_medications	Numerical
number_outpatient	Numerical
number_emergency	Numerical
number_inpatient	Numerical
diag_1	Nominal*
diag_2	Nominal*
diag_3	Nominal*
diag_4	Numerical
number_diagnoses	Numerical
Medications (15)	Nominal
change	Nominal
diabetesMed	Nominal
readmitted	Nominal

TABLE I: Original features and their data type. Those marked with an * were written as numbers but in reality is just a way to encode the categories. Decoding to categorical data has been done thanks to table 2 in [1] and the descriptions available in [2].

Next, we examined the *NaN* values in our dataset. Our initial idea was to eliminate all rows containing a *NaN*, but this would have reduced the dataset from 5,000 rows to just over 4,000. Instead, we decided to replace the *NaN* values with random samples drawn from the same feature they belong to, ensuring that the replacements follow the

* adam-cesar.mackowiak1@estudiant.uib.cat

original probability distribution of each column.

We then decided to eliminate certain features to reduce dimensionality:

- *Unnamed: 0* — This column did not provide any information beyond the row number.
- *chlorpropamide, glimepiride, acetohexamide, pioglitazone, rosiglitazone, troglitazone, tolazamide, examide, glyburide-metformin, glipizide-metformin, metformin-rosiglitazone* — These are medical drugs that had over 90% of their values labeled as *No*, making them highly imbalanced.
- *tolbutamide* — This feature was removed because 80% of its values were *NaN*.

Some of these drug-related features had 100% of their entries labeled as *No*, contributing no new information. Others had 99% *No* and 1% *Steady*, while a few had slightly more variation but still over 90% *No*. This approach can be criticized as being less rigorous than other methods. One might argue that a more principled strategy would involve removing only those features with 100% *No*, and then letting feature selection techniques determine whether the remaining features should be discarded. Moreover, since the dataset involves medical information, one could argue that all data is valuable, especially when it relates to human lives. Nevertheless, after discussions with my colleagues, we found that our classification scores, as well as the algorithm's performance, remained very similar despite these removals.

After closely inspecting several features, we decided to further reduce dimensionality by regrouping some categorical features, specifically `race`, `discharge_disposition_id`, and `admission_source_id`.

- *Race: Hispanic* and *Asian* were grouped into *Other*, since all three combined represented only 4% of the dataset.
- *Discharge Disposition ID: Hospice* and *Expired* were merged into a single category labeled *Other*, as together they accounted for just 3% of the data.
- *Admission Source ID: Transfer* and *Birth* were grouped into *Other*. *Birth* represented less than 1%, and *Transfer* made up only 6% of the dataset.

On the other hand, we decided to convert the feature *age* from nominal to numerical. Extreme caution is advised when making such transformations, as it is easy to introduce information that does not truly exist in the data. However, in this case, because the variable represents age, the conversion is relatively straightforward. We replaced the 10 age categories with 10 corresponding numerical values ranging from 0 to 0.9, such that $[0\text{–}(x+1)0] \to 0.x$.

Next, we applied One-Hot Encoding to all categorical features. These are: *race, gender, admission_type_id, discharge_disposition_id, admission_source_id, diag_1, diag_2, diag_3, citoglipton, glyburide, insulin, change, diabetesMed, readmitted*. This increased the dimensionality of our dataset to 52 features—still a manageable number, thanks to our earlier efforts in data exploration and dimensionality reduction.

Before proceeding with feature selection, we would like to briefly comment on two additional points.

First, while we computed a correlation matrix as part of our data exploration protocol, we did not extract any relevant insights from it for our final dataset. Additionally, we were hesitant to eliminate features based solely on the correlation matrix, as it only captures linear relationships, whereas machine learning algorithms are designed to detect both linear and nonlinear patterns.

On the other hand, there are two features that do not appear in either [1] or [2]. These are *citoglipton* and *diag_4*. Initially, we considered removing both features, but ultimately we decided to let the feature selection algorithms determine their relevance.

In the first case, *citoglipton*, some preliminary research suggested that it is similar to *sitagliptin*, a medical drug that does appear in the original dataset. According to our findings, both drugs seem to have similar properties, though they are distributed in different countries. However, since we are not medical experts, we again chose to leave the decision to the feature selection process.

The second case, *diag_4*, is more complicated. At first, we were inclined to remove it, assuming it was a categorical feature like *diag_1*, *diag_2*, and *diag_3*, which follow the ICD9 coding system with values ranging from 0 to 1000. However, *diag_4* includes values up to around 1500, which initially made us skeptical. Moreover, it has 5,000 unique values—the same as the number of rows in the dataset—whereas the other diagnosis features have around 370 unique values each. This indicates that *diag_4* is actually numerical rather than categorical. Further supporting this, the feature contains a high number of decimal places—often with a level of precision that, frankly, seems excessive. A measurement (which is what this appears to be) should not reasonably require precision to 10–12 decimal places.

In the end, we chose to retain *diag_4* for the time being and rely on the feature selection process to determine whether it is useful.

III. FEATURE SELECTION

We first rescaled all data using Min-Max scaling. Then, for feature selection, we decided to apply a Random Forest Classifier and conducted two different tests: the first was based on the mean decrease in impurity (Figure 1), and the second on the mean decrease in accuracy after permutation (Figure 2), both of which are explained in [3].

Three comments can be made regarding figures 1 and 2. The first is that both seem to exhibit a natural

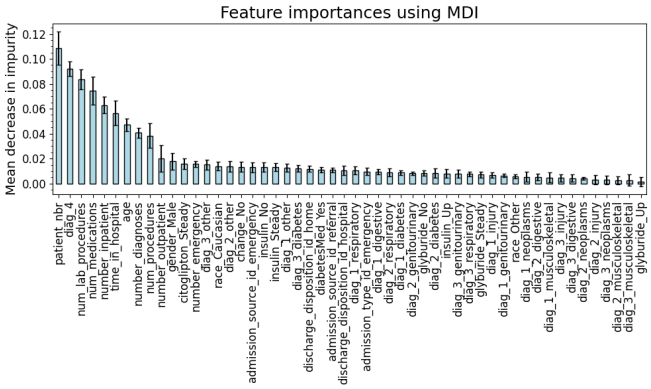


FIG. 1: Feature selection using mean decrease in impurity.

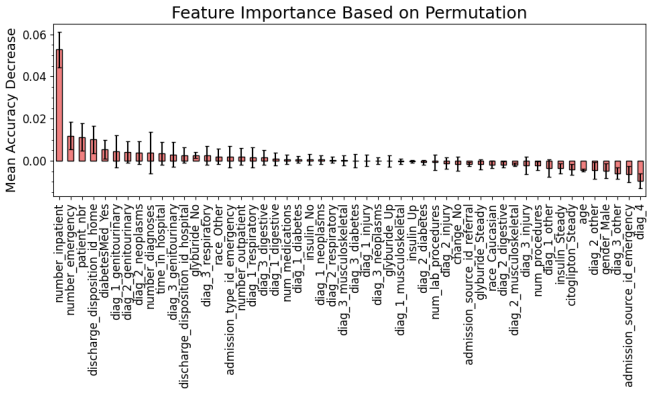


FIG. 2: Feature selection using mean accuracy decrease.

threshold for feature importance. In the case of MDI, the threshold appears to be around *num_procedures*, while for the permutation-based test, it seems to lie around *discharge_disposition_id_home*.

The second observation is that, in the feature importance test based on the mean decrease in accuracy, there are several features that, when permuted, appear to increase the model's accuracy rather than decrease it. These features could be considered noisy or potentially affected by multicollinearity, and thus removing them might be beneficial.

Nevertheless, we decided to combine the results from both tests for our feature selection process. Initially, we had 52 features. Based on this, the feature with the highest MDI score was assigned 52 points, the second highest 51 points, and so on. We applied the same scoring method to the permutation importance test. We then summed the scores from both methods to compute a final score for each feature, resulting in a combined ranking of feature importance.

The third and last observation is that, for both of these tests, we divided the dataset with 80% for the training part and 20% for the test part. One may say that we should have done a k-fold cross-validation, and, indeed, is not a bad idea. Nevertheless, we decided to do it this way.

Finally, we evaluated the model's accuracy as a function of the number of features included, in order to determine the optimal number of parameters to retain.

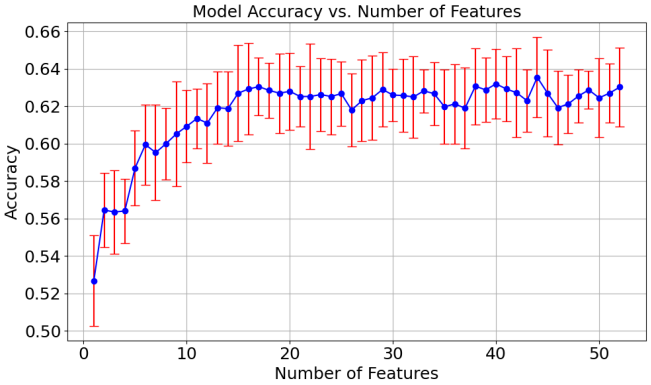


FIG. 3: Accuracy as we increase the number of parameters. We used a Top-Down approach.

We observed that with around 20 features, the model performance has reached already a plateau. Therefore, we decided to apply the machine learning algorithms using the 20 most important features, based on the previously described ranking. One could attempt to use even fewer features and, as shown in Figure 3, still obtain similar results.

A potential criticism of this approach is that we assigned equal weight to both importance tests. While this is true, no compelling reason has been identified to justify a different weighting scheme.

Finally, it is worth noting that neither *diag_4* nor *citoglipton* appeared among the selected features.

IV. MODELLING

We applied multiple machine learning algorithms to predict whether a patient will be readmitted within 30 days. These algorithms include K-Nearest Neighbors (KNN), Decision Tree, Random Forest, Naive Bayes, Support Vector Machine (SVM), Logistic Regression, and XGBoost. For each of these, we analyzed the classification score as a function of a selected model parameter.

To evaluate the models, we used 10-fold cross-validation. Additionally, for each model and parameter setting, we measured the time required to perform the 10-fold cross-validation repeated 100 times. This repetition was done to reduce noise and obtain more stable timing results. These timing comparisons help identify which algorithms are more computationally expensive and which parameters have a significant impact on runtime.

All corresponding plots are provided both in the code and in the appendix. The optimal parameters for each model are summarized in Table II.

	Optimal Parameter	Value
Random Forest	n_estimators	31
Decision Tree	max_depth	4
KNN	n_neighbors	8
SVC	kernel	poly
Logistic Regression	C	7.879000
XGBoost	n_estimators	21

TABLE II: Table that presents the optimal parameters for each model. As you can see, the Naive Bayes model does not appear, as it has no tunable parameters. For all other models, the remaining parameters are the default ones from their respective functions, except for the Random Forest and XGBoost models. In these cases, we chose to set the maximum depth of each tree to the optimal value found in the Decision Tree study. However, it should be noted that this may not necessarily be the optimal depth for the trees in the Random Forest and XGBoost models.

V. RESULTS

After finding the optimal values, we computed the accuracy, cost for each model using the cost matrix (Table III), and AUC using 10-fold cross-validation for each model with the optimal parameters. The results are displayed in Figure 4. Additionally, we also computed the ROC curve and have displayed it in Figure 5.

	Predicted Positive	Predicted Negative
Actual Positive	0 (TP)	50 (FN)
Actual Negative	10 (FP)	0 (TN)

TABLE III: Cost Matrix for Classification

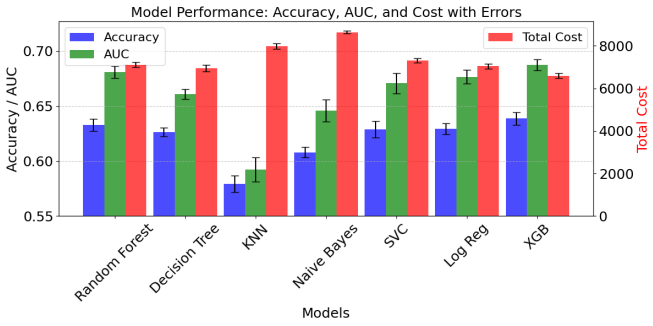


FIG. 4: Score for the AUC and accuracy and the total cost for each model.

As we can see, KNN performs the worst among all the models, followed by the Naive Bayes approach. However, the score achieved by the latter is surprising, as it assumes independence between features. On the other hand, the models that scored the best in terms of accuracy and AUC are XGBoost and Random Forest. It was also surprising to see Logistic Regression perform similarly to these two models.

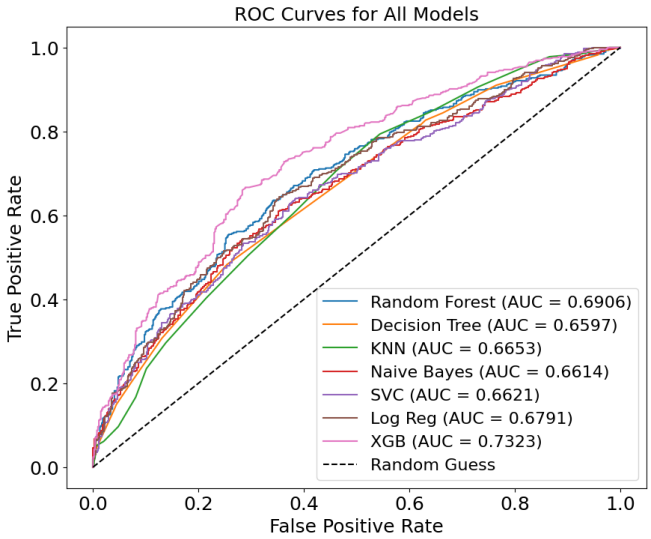


FIG. 5: ROC curve for each model. As we can see the biggest AUC for this case are the ones from XGBoost and Random Forest, which makes sense since XGBoost can be thought as an optimized Random Forest.

From a cost perspective, we aim to minimize the cost as much as possible, particularly given the importance of human lives. The models that appear to have the lowest cost are XGBoost, Decision Tree, and Random Forest. Considering all these factors, the best models seem to be XGBoost, followed by Random Forest, which makes sense, as XGBoost is an optimized version of Random Forest. Also, looking at the ROC curve, seems that XGB has a region of the curve where it has between 0.6 and 0.7 of true positive rate while retaining around 0.3 of false positive rate. Some will prefer to reduce even more the ratio of false positives, but this will reduce the true positive rate. Either way, XGB seems to have the best ROC curve.

Finally, shown in section B, we can also see that XGB was quite fast taking also into account its accuracy.

VI. CONCLUSIONS

This work demonstrates that classical machine learning algorithms, particularly XGBoost and Random Forest, can effectively predict hospital readmissions when supported by meticulous data preprocessing and feature selection. Despite the simplicity of some models, like Naive Bayes or Logistic Regression, their performance exceeded expectations, giving good accuracy and AUC when compared to other models for this dataset. However, the most balanced models across accuracy, AUC, and cost were XGBoost and Random Forest, with XGBoost slightly outperforming the rest. The total cost

and the ROC curve further emphasizes the importance of minimizing false negatives due to their critical implications in healthcare scenarios. Finally, XGB seems to be the best model for this job, taking into account its high classification score, its low total cost and the fact that it's quite fast. Nevertheless, across all models, the classification scores are really, which can be due to the complex nature of this task. Applying these models to a larger sample would be a valuable next step. Additionally, a comparison with Deep Learning techniques is advisable.

ACKNOWLEDGMENTS

I would like to thank Massimiliano Zanin, my colleagues and Ismael García Valentín, a friend of mine who studied Computer Science, for giving their time to discuss the best approaches for this project.

[1] B. Strack, J. P. DeShazo, C. Gennings, J. L. Olmo, S. Ventura, K. J. Cios, and J. N. Clore, Impact of hba1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records, BioMed research international **2014**, 10.1155/2014/781670 (2014).

[2] C. K. D. J. Clore, John and B. Strack, Diabetes 130-US Hospitals for Years 1999-2008, UCI Machine Learning Repository (2014), DOI: https://doi.org/10.24432/C5230J.

[3] S. learn developers, Feature importances with a forest of trees (2024), accessed: 2025-04-06.

Appendix A: Decoding of id and icd9

Feature	Original Values
admission_type_id	{1, 2, 7: emergency; 3: elective; 4: newborn; 5, 6, 8: NaN}
discharge_disposition_id	{1, 6, 8: home; 11: expired; 13, 14: hospice; 2, 3, 4, 5, 7, 9, 10, 15-29: hospital; 18, 25, 26: NaN}
admission_source_id	{1, 2, 3: referral; 4, 5, 6, 10, 18, 22, 25, 26: transfer; 7: emergency; 8: law_enforcement; 11-14, 23, 24: birth; 19: readmission; 9, 15, 17, 20, 21: NaN}

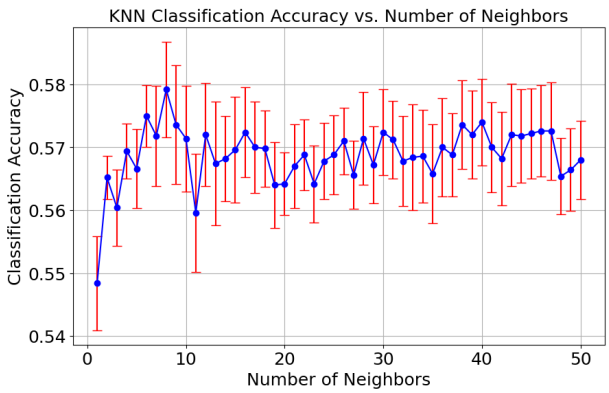
TABLE IV: Decoding and reduction of dimensionality of the *id* features.

Appendix B: Plot of accuracy and time for each model

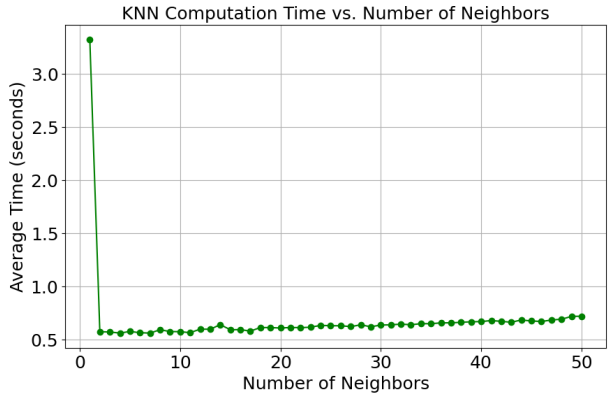
As we can see, there are different behaviors for each model, which makes sense. One thing should be clarified about the way the time is measured. As the reader can see, the y-axis is labeled "average time," except for *SVM*. This label can be a bit misleading, so we want to explain in more detail what we did. Initially, we measured the time it took for the code to perform the 10-fold cross-validation for each model and divided it by 10, so that we got the mean time per fold. The issue is that the running times were too short and were heavily influenced by noise. The noise here is due to the randomness of the computer deciding to perform other tasks instead of running our models, as well as the inherent randomness of some models. Because of this, we decided to run the *cross_val_score* function 100 times and saved the time of the last 10-fold cross-validation. The problem is that we did not change the part where we divided the time by 10. So, in reality, the actual computation time is not what we plotted, but rather one order of magnitude higher. Still, all of this was done simply to compare the times and scores for different parameters and models.

One could try running the 10-fold cross-validation one thousand times instead of one hundred to make the plots more robust to noise, but this would take a lot more time. This sensitivity to noise can be seen with *Logistic Regression* and, to a lesser extent, with *Decision Tree*. We also observe something very curious with *KNN* and *XGB*, where the initial parameter results in a long computation time, then the time sharply decreases, and then it starts to increase again. This strange behavior could be due to some kind of approximation performed by the algorithm once the parameter surpasses a certain threshold or maybe is related to something related with the computer where the tests have been done, but we cannot say for sure.

Finally, as discussed in Section V, the best results seem to be given by *XGB*. Moreover, if we look at the optimal parameter for the score, the time the model takes is also very short. Therefore, as we concluded, *XGB* seems to be the best machine learning model for this dataset and this task.

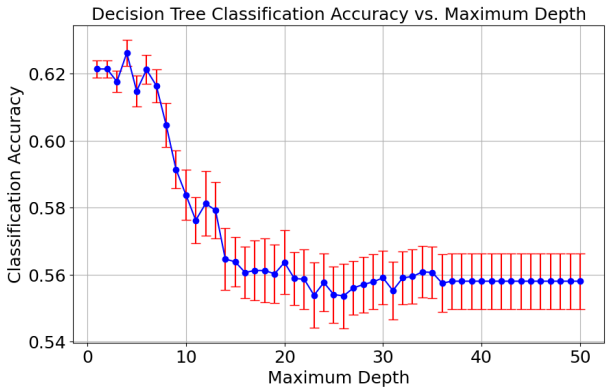


(a) Score as we change the number of neighbors

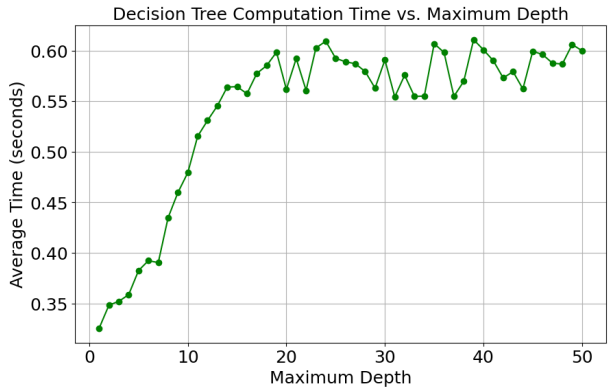


(b) Time as we change the number of neighbors

FIG. 6: KNN

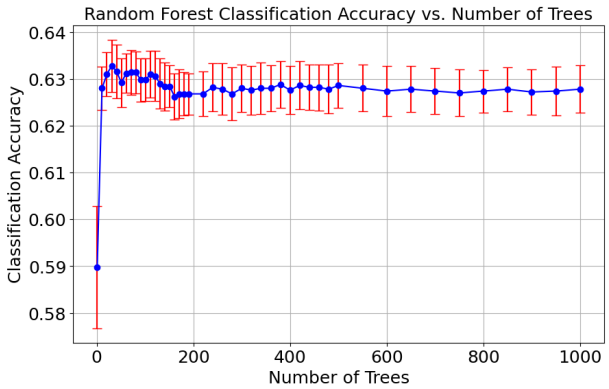


(a) Score as we change the maximum depth

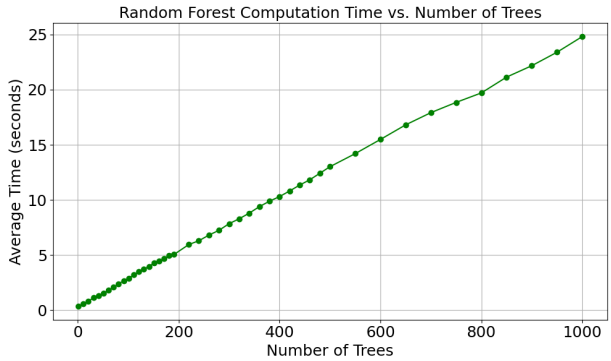


(b) Time as we change the maximum depth

FIG. 7: Decision Tree

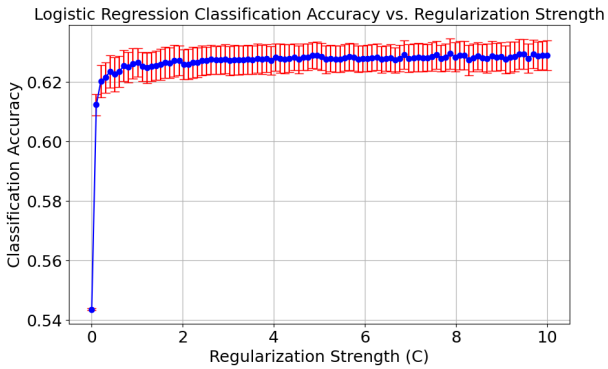


(a) Score as we change the number of trees with maximum depth 4

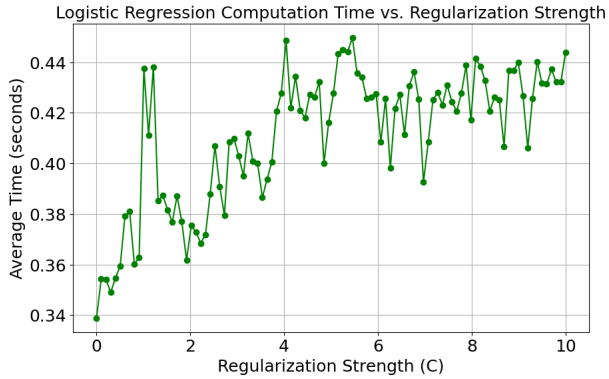


(b) Time as we change the number of trees with maximum depth 4

FIG. 8: Random Forest

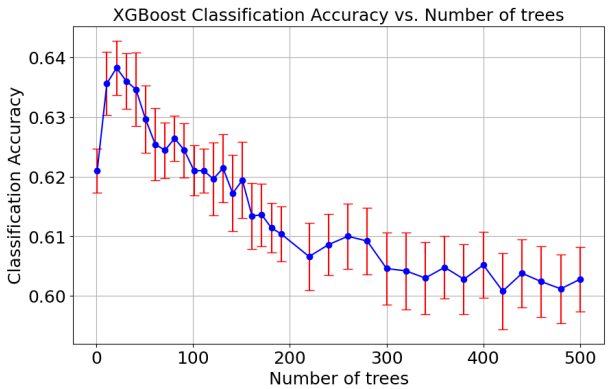


(a) Score as we change the regularization strength C

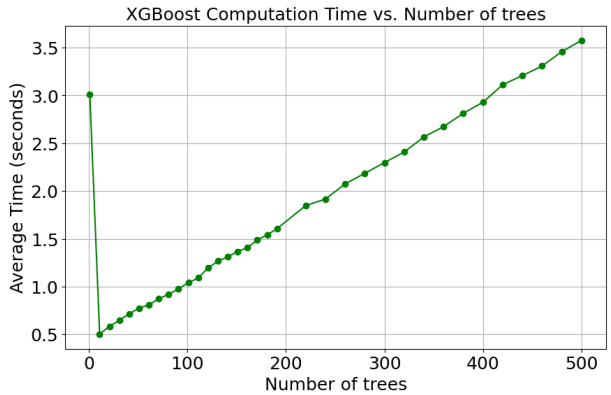


(b) Time as we change the regularization strength C

FIG. 9: Logistic Regression



(a) Score as we change the number of tress with maximum depth 4



(b) Time as we change the number of tress with maximum depth 4

FIG. 10: XGB

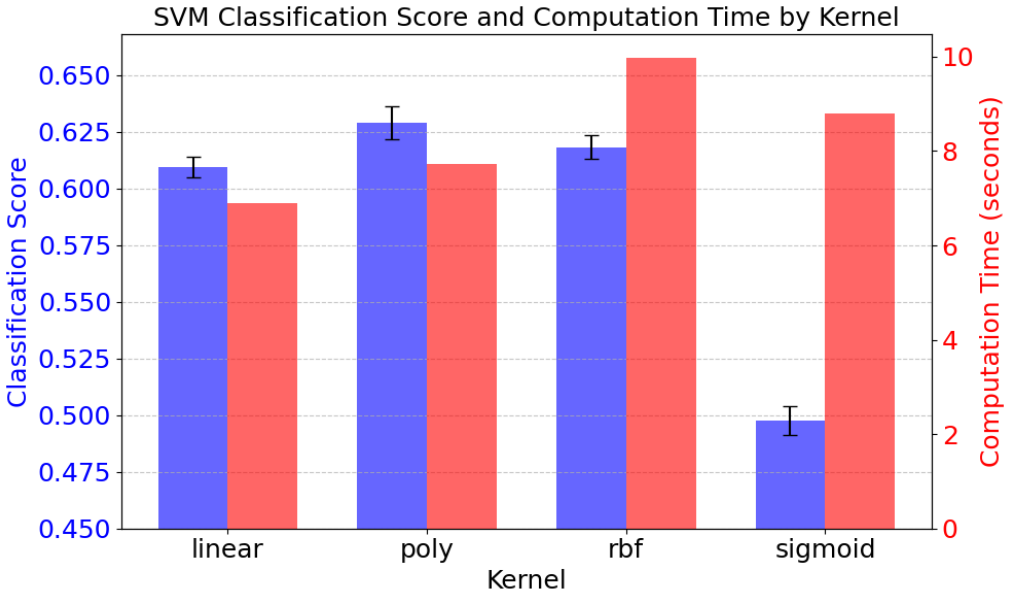


FIG. 11: Score and time for different kernels for SVM.